

Rapport du Système de Recommandation Glamourly

1. Vue d'ensemble

Le projet **Glamourly** est une plateforme e-commerce de produits de beauté haut de gamme. Nous avons implémenté un moteur de recommandation de classe production qui combine plusieurs stratégies pour maximiser l'engagement utilisateur et les conversions.

2. Architecture Technique

- **Frontend** : React.js (Vite) avec Tailwind CSS pour un design moderne et responsive.
- **Backend** : Python (FastAPI) pour une API haute performance.
- **Base de données** : SQLite (via SQLAlchemy ORM) pour une gestion fluide des données produits, utilisateurs et interactions.

3. Structure du Projet

```
/backend
/app
  main.py          # Points d'entrée de l'API et logique de routage
  models.py        # Schémas de la base de données (SQLAlchemy)
  recommendation.py # Cœur du moteur de recommandation (Algorithmes)
/frontend
/src
  /components      # Composants réutilisables (Navbar, ProductCard, RecommendationSection)
  /pages           # Pages de l'application (Home, ProductDetails, Cart, Login)
  /services        # Communication avec l'API (Axios)
```

4. Système de Recommandation

Le moteur utilise un algorithme hybride avancé basé sur trois piliers :

A. Personnalisation & Comportement (Behavioral)

- **Historique d'achat** : Analyse les catégories précédemment achetées pour booster les produits similaires.
- **Produits consultés** : Suit les consultations en temps réel pour suggérer des produits pertinents durant la session.
- **Liste de souhaits (Wishlist)** : Utilise les articles sauvegardés comme signaux forts d'intérêt.
- **Calcul de score** : Chaque interaction reçoit un poids spécifique, avec une décomposition exponentielle du score basée sur la récence (les actions récentes comptent plus que les anciennes).

B. Intelligence Produit (Content-based)

- **Similarité de contenu** : Recommande des produits basés sur la catégorie et la proximité de prix.
- **Fréquemment achetés ensemble** : Analyse les commandes pour suggérer des produits complémentaires (cross-selling).

C. Classement & Ranking (Smart Scoring)

- **Tendance (Trending)** : Met en avant les produits ayant le plus grand nombre de vues sur les 7 derniers jours.
- **Nouveautés** : Applique un boost temporaire aux produits récemment ajoutés au catalogue.
- **Gestion du démarrage à froid (Cold Start)** :
 - Pour les nouveaux utilisateurs : Affiche les best-sellers et les tendances générales.
 - Pour les nouveaux produits : Utilise la similarité de contenu pour les intégrer aux recommandations.

5. Modèles de Données

- **Product** : Nom, catégorie, prix, image, description, vues, date de création.
- **User** : Identifiants et catégorie préférée.
- **ProductView** : Journalisation de chaque vue produit par utilisateur.
- **WishlistItem** : Articles mis en favoris.
- **Order / OrderItem** : Historique complet des transactions pour le filtrage collaboratif.

6. Guide d'Installation et Lancement

Backend

1. Naviguez vers le dossier backend : `cd backend`
2. Installez les dépendances : `pip install -r requirements.txt`
3. Lancez le serveur : `uvicorn app.main:app --reload`
4. L'API sera disponible sur `http://localhost:8000`

Frontend

1. Naviguez vers le dossier frontend : `cd frontend`
2. Installez les dépendances : `npm install`
3. Lancez l'application : `npm run dev`
4. Ouvrez `http://localhost:5173` dans votre navigateur

7. Guide de Déploiement

- **Backend** : Peut être déployé sur Heroku, AWS Lambda ou un VPS utilisant Gunicorn/Uvicorn.
- **Frontend** : Déploiement recommandé sur Vercel, Netlify ou AWS S3+CloudFront.
- **Base de données** : Pour la production, remplacez SQLite par PostgreSQL.